

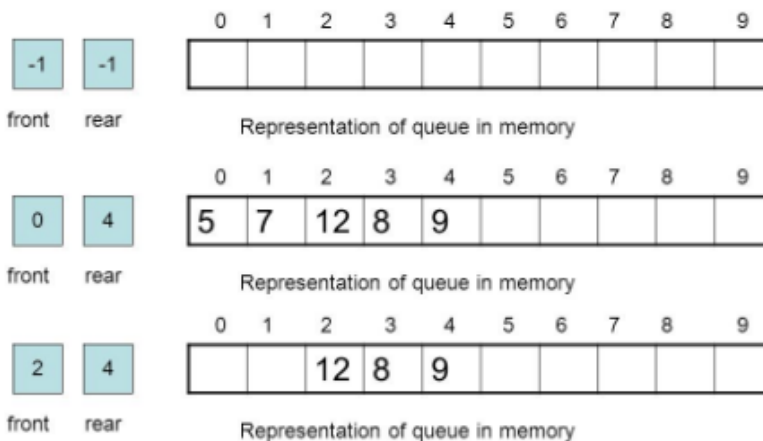
Queue

- *A queue is an ordered collection of items where an item is inserted at one end called the “rear” and an existing item is removed at the other end, called the “front”.*
- Queue is also called as FIFO list i.e. First-In First-Out.
- In the queue only two operations are allowed enqueue and dequeue.
- Enqueue means to insert an item into back of the queue.
- Dequeue means removing the front item. The people standing in a railway reservation row are an example of queue.

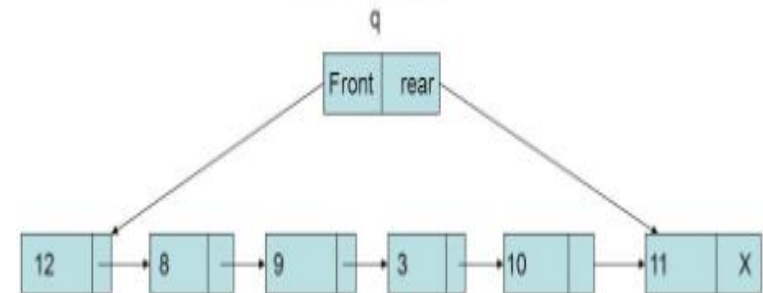
Queue

- The queue can be implemented into two ways:
 - Using arrays (Static implementation)
 - Using pointer (Dynamic implementation)

Array representation of linear queue



Representation of a queue in memory



Types of Queues

- Queue can be of four types:
 - Simple Queue
 - Circular Queue
 - Priority Queue
 - De-queue (Double Ended Queue)

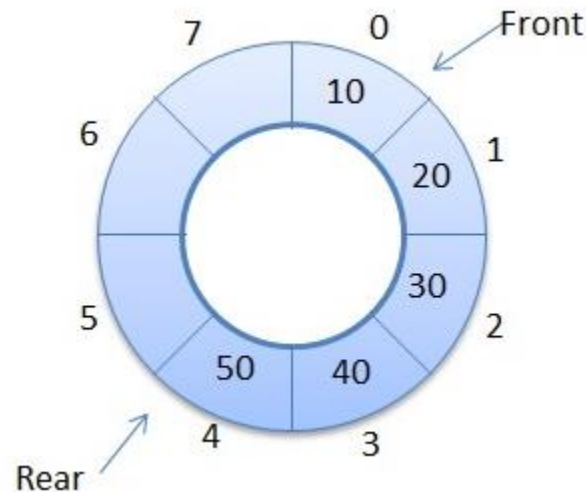
Simple Queue

- Simple Queue: In simple queue insertion occurs at the rear end of the list and deletion occurs at the front end of the list.



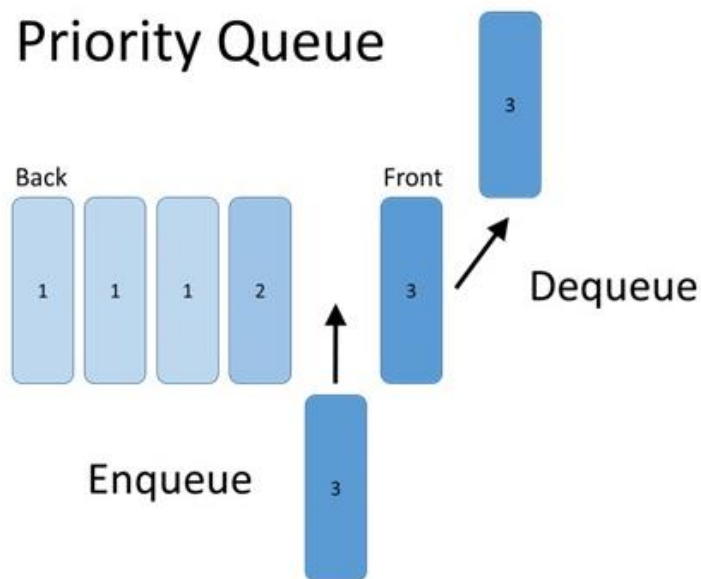
Circular Queue

- Circular Queue: A circular queue is a queue in which all nodes are treated as circular such that the last node follows the first node.



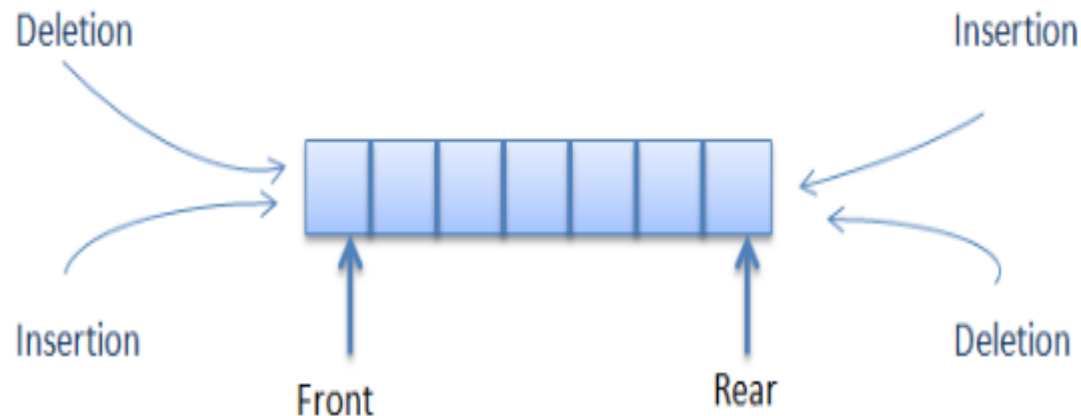
Priority Queue

- **A priority queue is a queue that contains items that have some present priority. An element can be inserted or removed from any position depending upon some priority.**



Deque Queue

- Dequeue: It is a queue in which insertion and deletion takes place at the both ends.



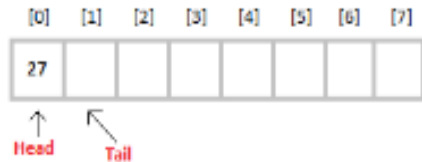
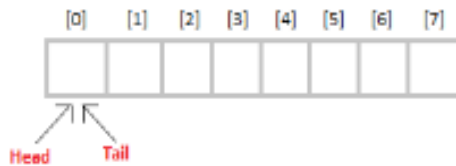
Operation on Queues

- **Queue()**: It creates a new queue that is empty.
- **enqueue(item)**: It adds a new item to the rear of the queue.
- **dequeue()**: It removes the front item from the queue.
- **isEmpty()**: It tests to see whether the queue is empty.
- **size()**: It returns the number of items in the queue.

Memory Representation of a queue using array

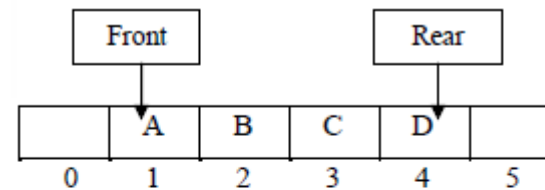
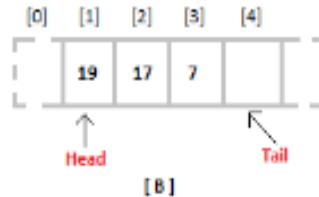
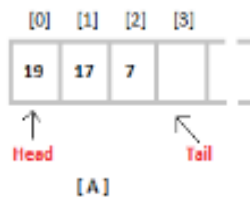
- Queue is represented in memory using linear array.
- Let QUEUE is a array, two pointer variables called FRONT and REAR are maintained.
- The pointer variable FRONT contains the location of the element to be removed or deleted.
- The pointer variable REAR contains location of the last element inserted.
- The condition $\text{FRONT} = \text{NULL}$ indicates that queue is empty.
- The condition $\text{REAR} = \text{N}-1$ indicates that queue is full.

Memory Representation of a queue using array

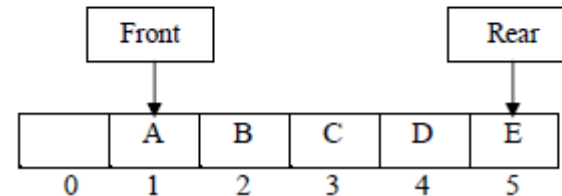


Adding elements to Queue

removing element from Queue



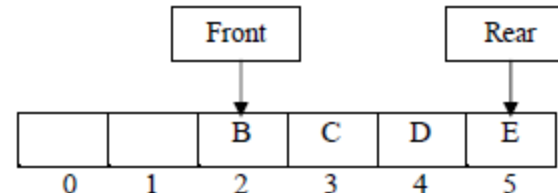
$REAR = REAR + 1, QUEUE[REAR] = 'E'$



Front = 1
Rear = 4

Front = 1
Rear = 5

$ITEM = QUEUE[FRONT], FRONT = FRONT + 1$



Front = 2
Rear = 5

Queue Insertion Operation (ENQUEUE):

- **ALGORITHM: ENQUEUE (QUEUE, REAR, FRONT, ITEM)**

QUEUE is the array with N elements. FRONT is the pointer that contains the location of the element to be deleted and REAR contains the location of the inserted element. ITEM is the element to be inserted.

Step 1: if REAR = N-1 then [Check Overflow]

PRINT "QUEUE is Full or Overflow"

Exit

[End if]

Q[0]	Q[1]	Q[2]	Q[3]	Q[4]
8	12	4	18	34

Step 2: if FRONT = NULL then [Check Whether Queue is empty]

FRONT = -1

REAR = -1

else

REAR = REAR + 1 [Increment REAR Pointer]

Q[0]	Q[1]	Q[2]	Q[3]	Q[4]

Q[0]	Q[1]	Q[2]	Q[3]	Q[4]
8	12			

Step 3: QUEUE[REAR] = ITEM [Copy ITEM to REAR position]

Step 4: Return

Queue Deletion Operation (DEQUEUE)

ALGORITHM: DEQUEUE (QUEUE, REAR, FRONT, ITEM)

QUEUE is the array with N elements. FRONT is the pointer that contains the location of the element to be deleted and REAR contains the location of the inserted element. ITEM is the element to be inserted.

Step 1: if FRONT = NULL then [Check Whether Queue is empty]

PRINT "QUEUE is Empty or Underflow"

Exit

[End if]

Step 2: ITEM = QUEUE[FRONT]

Step 3: if FRONT = REAR then [if QUEUE has only one element]

FRONT = NULL

REAR = NULL

else

FRONT = FRONT + 1 [Increment FRONT pointer]

Step 4: Return

Q[0]	Q[1]	Q[2]	Q[3]	Q[4]

Q[0]	Q[1]	Q[2]	Q[3]	Q[4]
8				

Q[0]	Q[1]	Q[2]	Q[3]	Q[4]
8	12			

Application of Queue

- Simulation
- Various features of Operating system
- Multi-programming platform systems.
- Different types of scheduling algorithms
- Round robin technique algorithms
- Printer server routines
- Various application software's is also based on queue data structure.